

Green - Fully Completed, Yellow - Partial Success, Red - Not Implemented,

Multi-Agent Competitive Intelligence Research System - Checklist (according to task document) - Bineet Ratna Shakya

1. Project Setup & Planning

- Review all requirements and estimate development time
- Initialize public GitHub repository
- Add basic folder structure (src/, agents/, utils/, etc.)
- Send repo link to subodh.shah@berrybytes.com after first commit
- Set up Python virtual environment with dependencies (requirements.txt)

2. Documentation (README.md)

- Add installation instructions - how to install dependencies and run the project
- Include architecture overview - describe agent roles and interaction flow
- Document agent design explanation - details of each agent's responsibilities
- List TO-DOs or known limitations - pending features or constraints

3. Agent Architecture Design

SearchAgent

- Define role - fetch top 5 relevant sources for a given topic
- Integrate search tool - use SerpAPI / DuckDuckGo / Requests-HTML
- Handle rate limiting - add retry logic, fallback if needed
- Format output - return list of URLs or search results

SummarizerAgent

- Define role - extract concise info from fetched content
- Use LLM for summarization - OpenAI, Hugging Face, Mistral
- Format output - product name, summary, source, date
- Handle missing fields gracefully - date optional, handle no content

VerifierAgent

- Define role - filter out unreliable or irrelevant updates
- Validate content - check for hallucinations or off-topic data
- Score relevance - optional scoring mechanism for filtering
- Return filtered list - cleaned-up updates only

CoordinatorAgent (Optional)

- Orchestrate workflow - manage agent communication
- Use LangChain/AutoGen - leverage built-in orchestration tools
- Handle retries/failures - gracefully retry failed steps

+1 BaseAgent (Custom)

4. Tool Integration

- Choose at least one external tool - e.g., SerpAPI, DuckDuckGo, Requests-HTML
- Implement tool integration - write wrapper functions or classes
- Handle rate limits - add sleep/retry logic where necessary
- Mock responses (if needed) - for testing or API limit scenarios

5. Logging & Memory

- Enable logging - log input/output of each agent step
- Implement short-term memory - in-memory state or LangChain memory
- Avoid redundant queries - cache or track previous queries

6. Output Generation

- Create structured output format - JSON or Markdown as specified
- Include required fields - product, update, source, date
- Save to file - generate report file per run
- Handle gracefully - missing or irrelevant data should not crash system

7. Testing & Evaluation Report

- Run test cases - with sample topics like "Notion AI"
- Evaluate outputs - check quality and structure
- Write evaluation report - 1–2 pages PDF/Markdown
 - Include system design choices - explain decisions made during development
 - Include challenges faced - technical hurdles encountered
 - Include suggestions for improvement - future enhancements

8. Bonus Tasks (Optional)

- Add guardrails - validate output format, filter hallucinations
 -
- Create basic UI - Gradio or Streamlit interface
- Implement fallback agent - dynamic web search when no data found
- Current Fallback Mechanisms (Built-in, not Agent-based):
 1. Search Tool Fallbacks:
 - Multiple extraction methods (Playwright → Newspaper3k → BeautifulSoup → Requests)
 - Multiple search strategies (general → news → blogs)
 2. Content Processing Fallbacks:
 - BART summarization → Simple extraction if BART fails
 - JSON parsing with multiple fallback approaches
 -
- Add prompt refinement agent - rewrite vague queries
 - Partially Implemented: Query Enhancement (not full Prompt Refinement Agent)
- Enable open retrieval from PDFs - allow document-based sources

9. Repository Finalization

- Add all source code - all modules and scripts
- Include sample outputs - example reports or test logs
- Finalize README.md - complete documentation
- Add design report - evaluation report in repo
- Make final commit - ensure everything is committed and pushed

10. Submission

- Submit by deadline - 25th July, 2025
- Ensure repo completeness - code, docs, examples, design report included